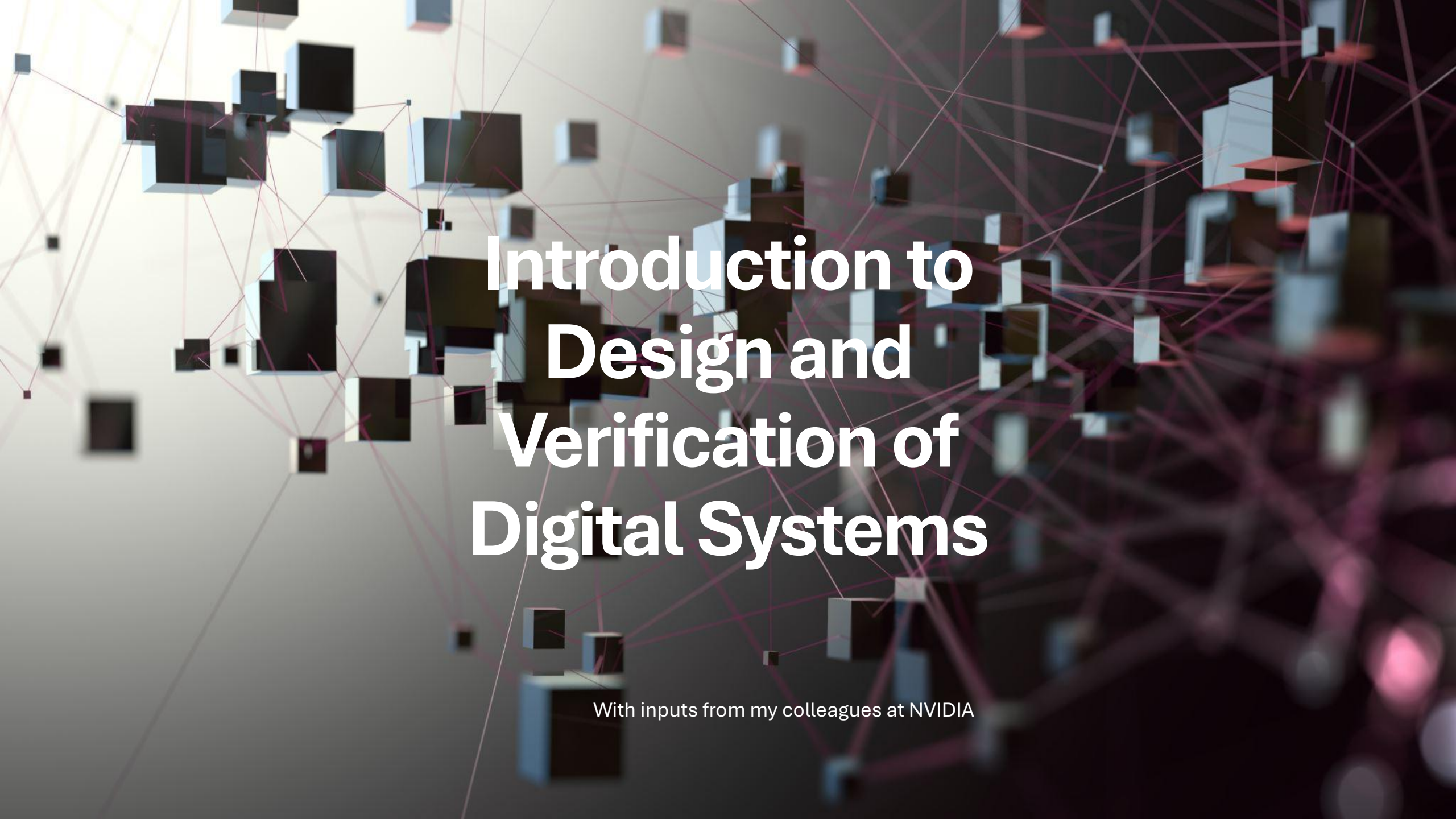


The background is a complex, abstract network of 3D cubes and red lines. The cubes are in various shades of grey, black, and white, and are connected by a dense web of thin red lines. The overall effect is a sense of a complex, interconnected system, possibly representing a digital network or a complex data structure.

Introduction to Design and Verification of Digital Systems

With inputs from my colleagues at NVIDIA

Logistics

- Pre-Req Test
 - Score won't count towards the grade.
 - Multiple attempts enabled.
- Travel
 - Lectures during the week of Sep 7 will be on Zoom or a recorded video will be released.
 - Please do not come to the classroom.
- Synopsys trainings
 - You might be seeing the trainings behind a paywall.
 - I've emailed Synopsys about this issue.
 - They replied saying everyone needs to log in first.

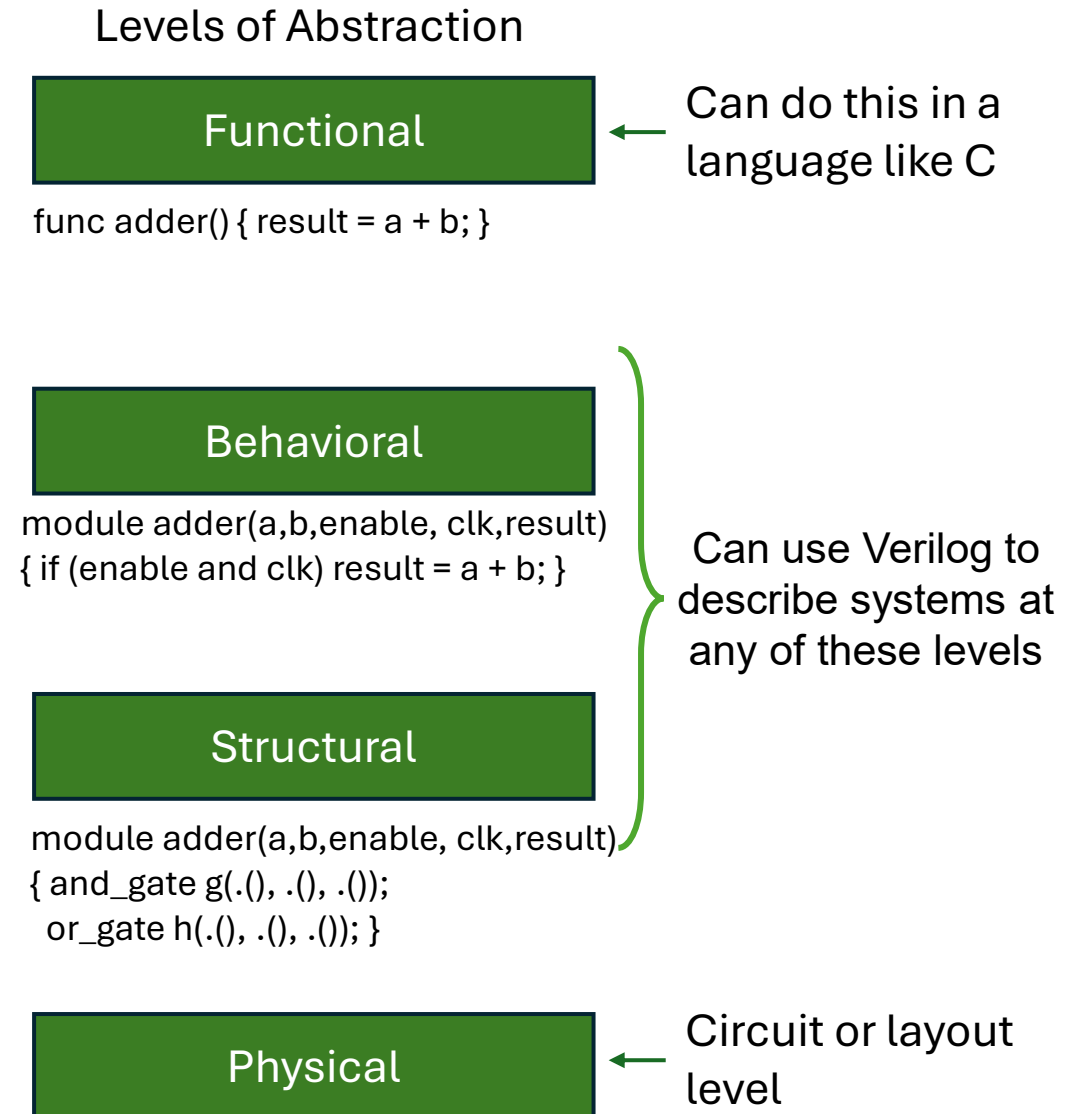
Kickoff Survey Results

On Canvas

Design

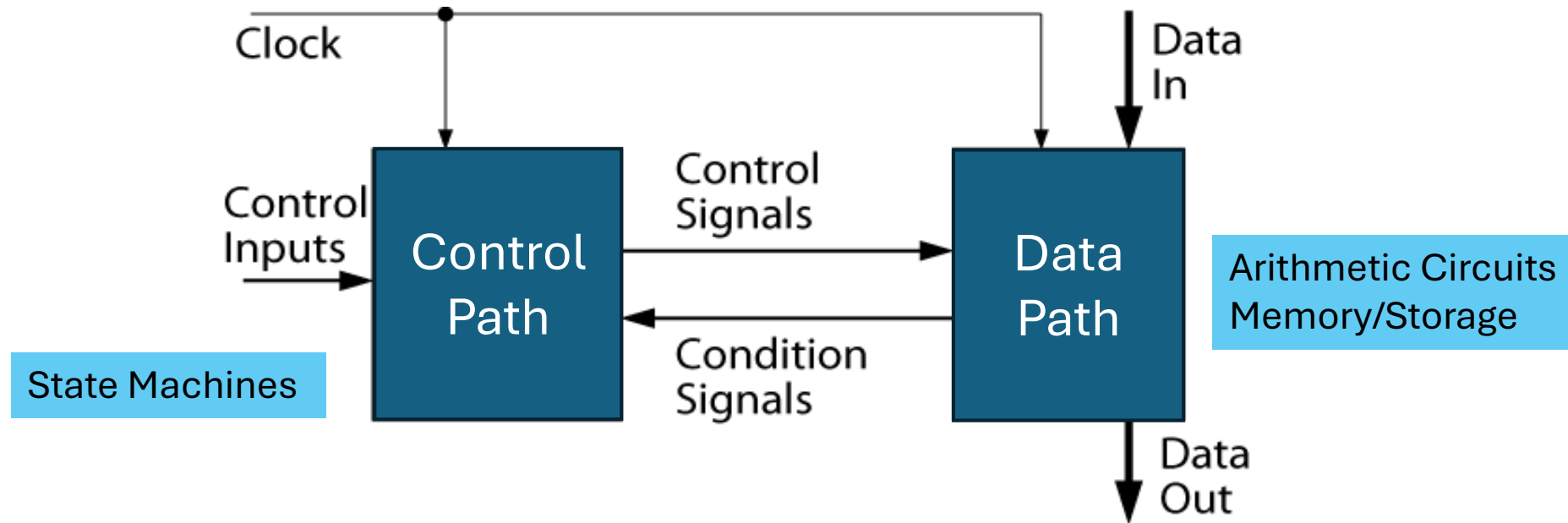
Abstraction Levels

- Register Transfer Level (RTL)
 - Describe the hardware as flow of data from one set of registers to next at each clock cycle
- RTL is somewhere in between behavioral and structural
 - Behavioral -> You may not be able to synthesize. Too abstract.
 - Structure (gate level) -> Too tedious to manually write. Generated by synthesis tools.
- All these terms are loosely defined

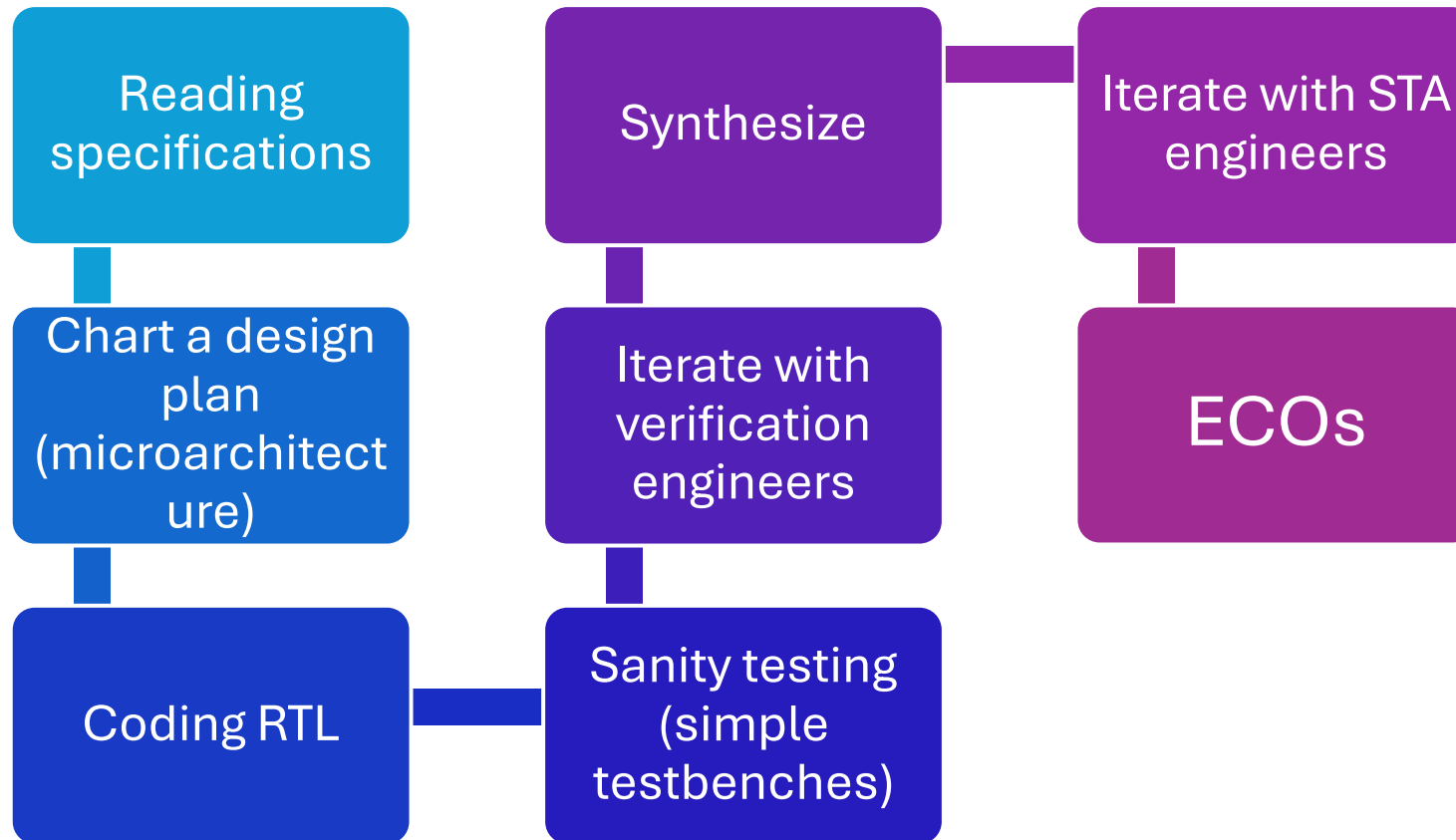


General View of Digital Systems

- Data Path (or Computation Unit)
- Control Path (or Controller)



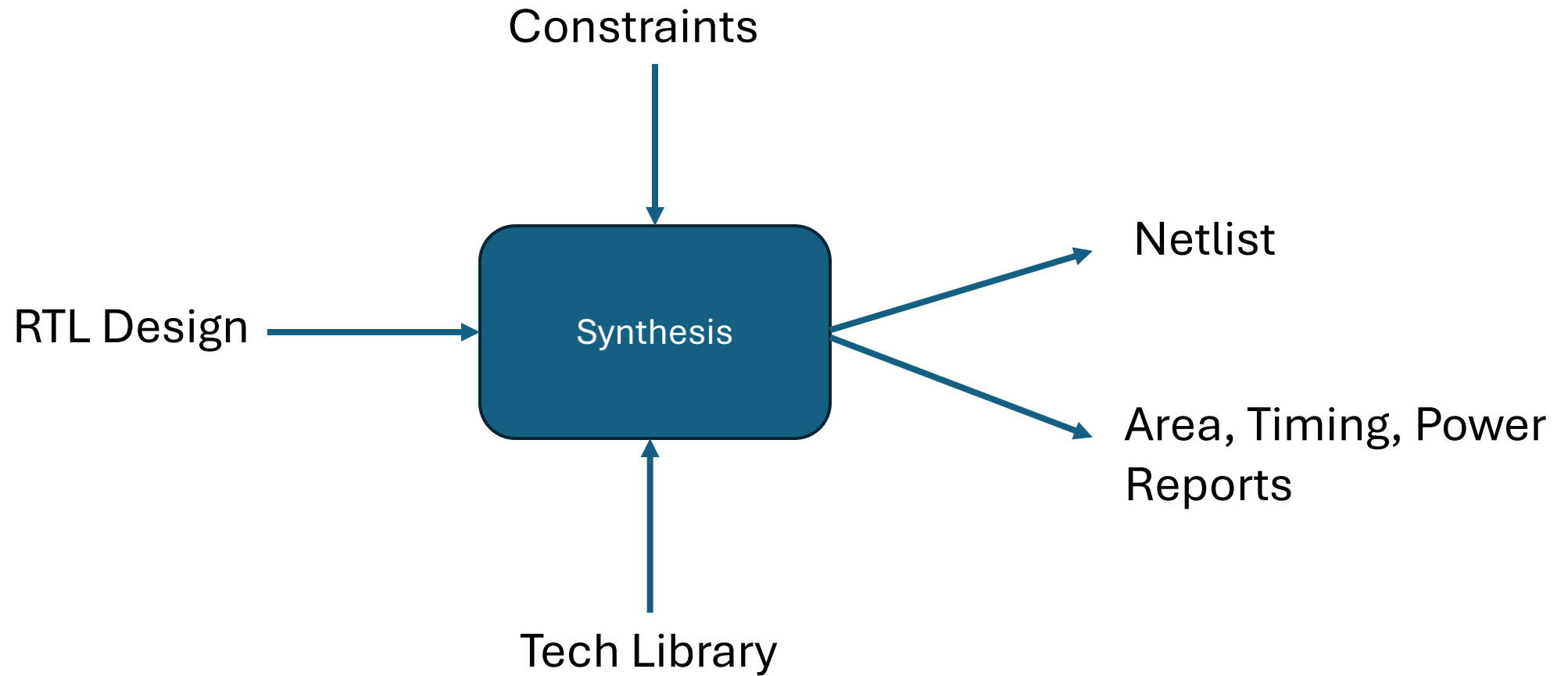
Life of a Design Engineer



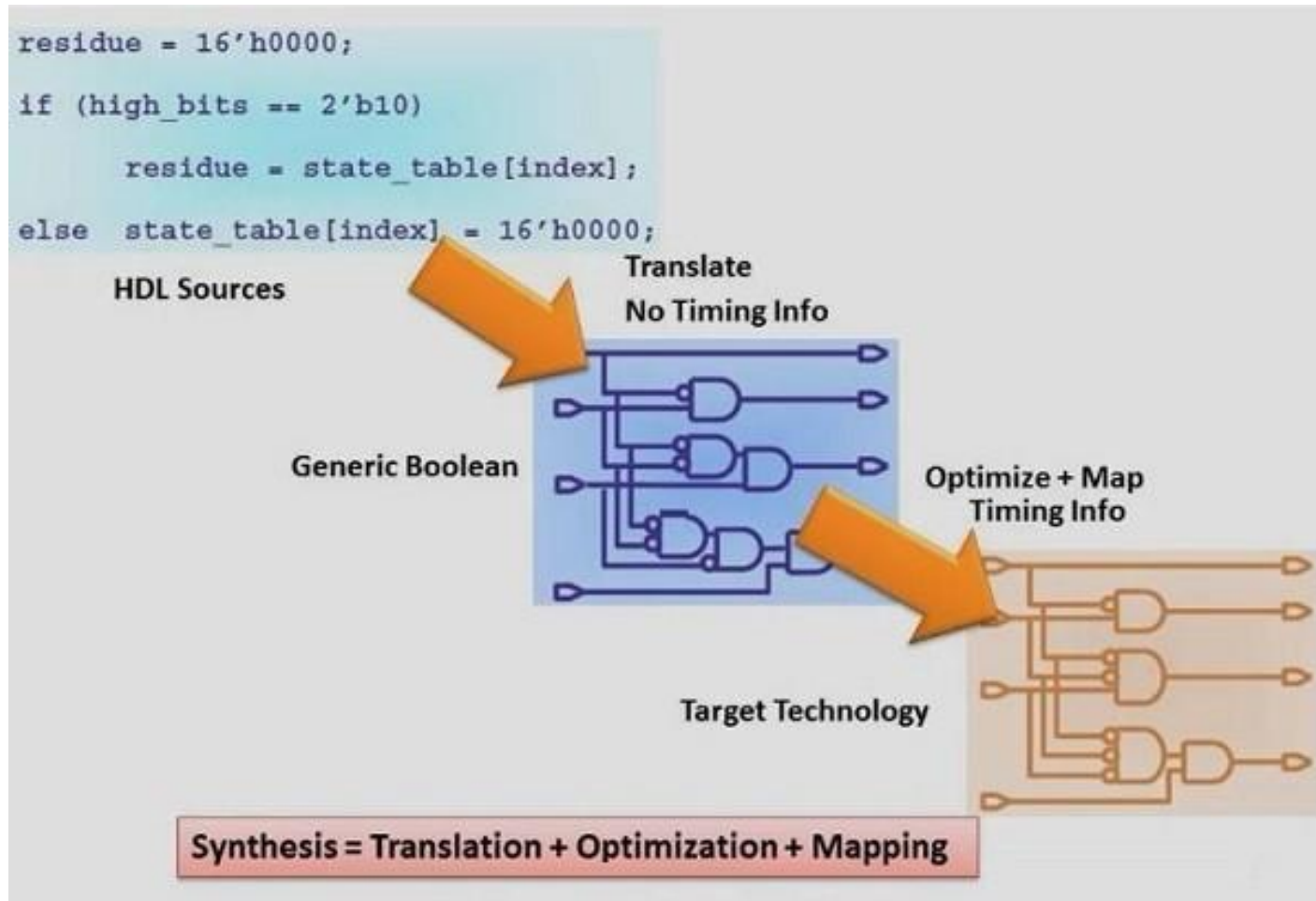
Synthesis

- Conversion of high-level HDL code to a netlist (**building blocks or cells** connected using wires)
- Building blocks are provided by the foundry
 - Called a “**tech library**” or “**standard cell library**”
 - Cells they can manufacture on a chip
- Building blocks are usually gates and flip-flops
 - “Standard cells”
- Example: Converts a “+” operator in the HDL code to the gates that constitute an adder
- Common tools: Synopsys Design Compiler, Cadence Genus

Synthesis



Synthesis



Static Timing Analysis

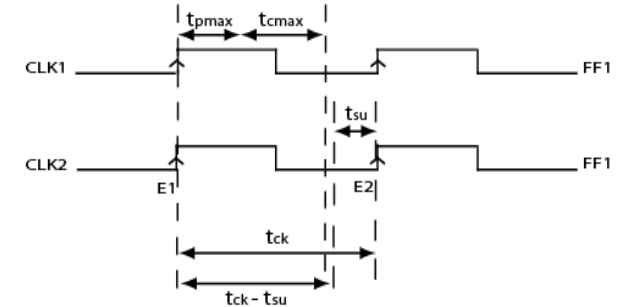
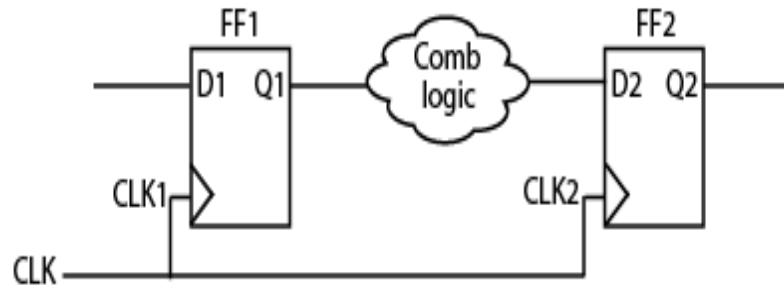
Test: What is “slack”?

- Method of validating the timing performance of a digital circuit by analyzing all timing paths.

- **Static..** Because no input vectors involved

- Timing paths

- PI to Flop
- Flop to Flop
- Flop to PO
- PI to PO



- Timing rules

- Data launched by FF1 on E1 of CLK1 should be captured by FF2 on E2 of CLK2
- Data launched by FF1 on E1 of CLK1 should not be captured by FF2 on E1 of CLK2

Timing Constraints

- Specify the required timing behavior
- Inputs to the synthesis tool
- What kind of constraints?
 - Clock period
 - Input and output delays
 - Timing exceptions (false paths, multicycle paths)
 - Clock uncertainties (e.g., clock skew)

```
create_clock -name CLK1 -period 10 [get_ports clka]  
create_clock -name CLK2 -period 12 [get_ports clk b]  
create_clock -name CLK3 -period 14 [get_ports clkc]  
  
set_input_delay 2 -clock [get_clocks CLK1] [get_ports in1]  
set_input_delay 1 -clock [get_clocks CLK2] [get_ports in2]
```

Source: EETimes

Technology Library

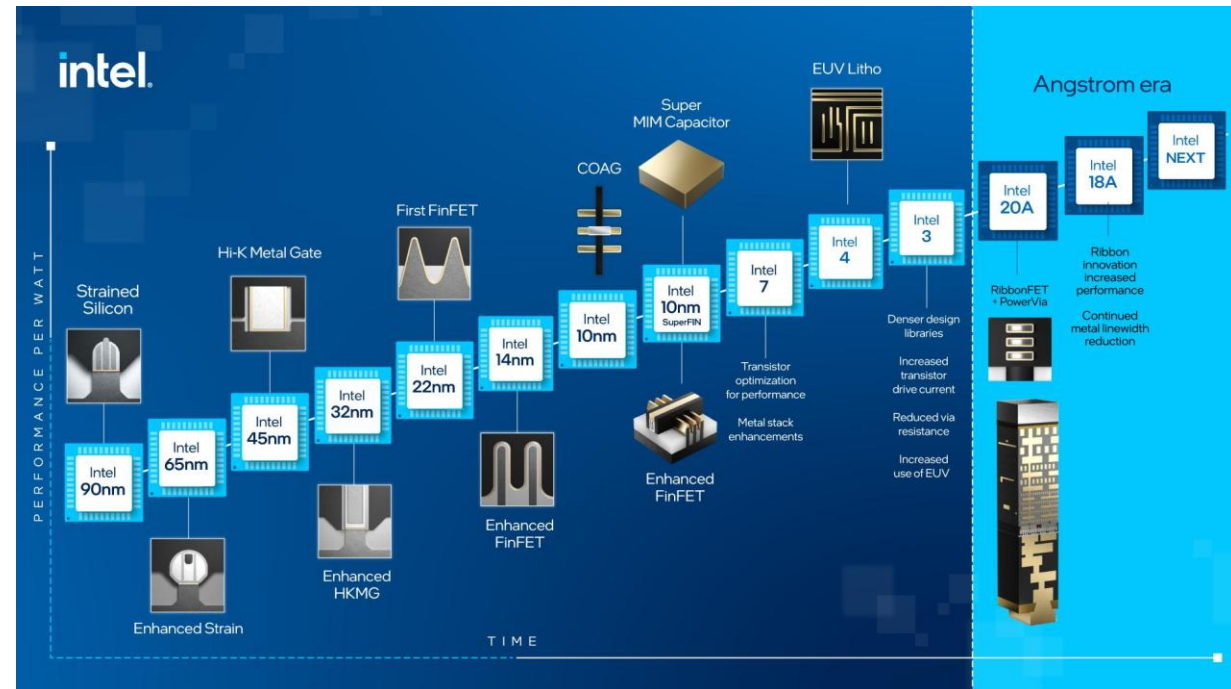
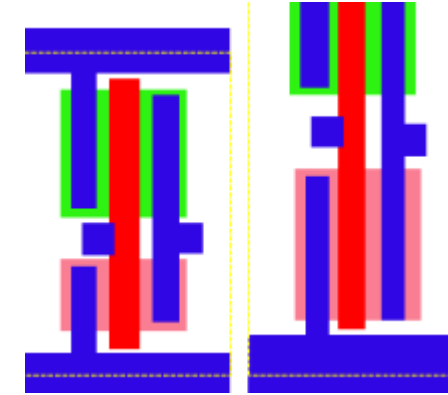
- A collection of pre-designed and pre-characterized “cells” in a specific semiconductor process.
- Also called standard cell library
- What type of cells?
 - Logic Cells: AND, OR, NAND, NOR, XOR, INV, Mux, AOI
 - Sequential Cells: Flip-Flops of different types, Latches
 - Special Cells: Buffers, Clock Gating Cells, Tie Cells
- What type of data?
 - Physical Data: Cell dimensions, pin locations
 - Timing Data: Delay, setup/hold times, slew rates
 - Power Data: Dynamic and leakage power

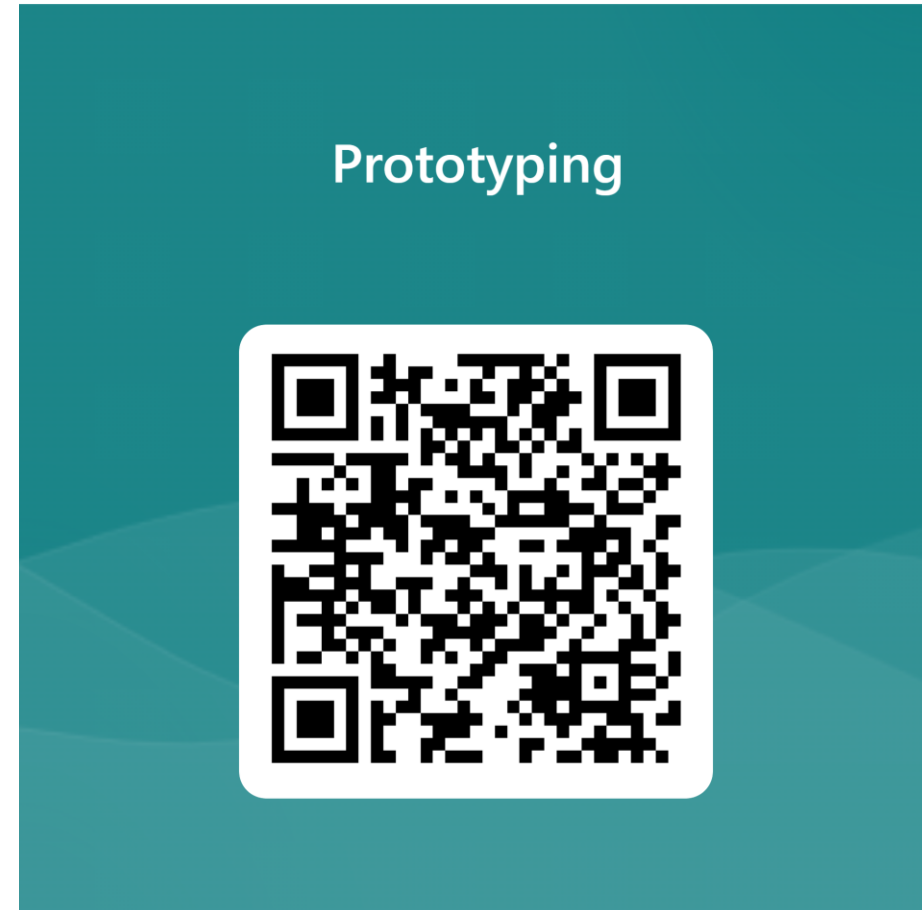
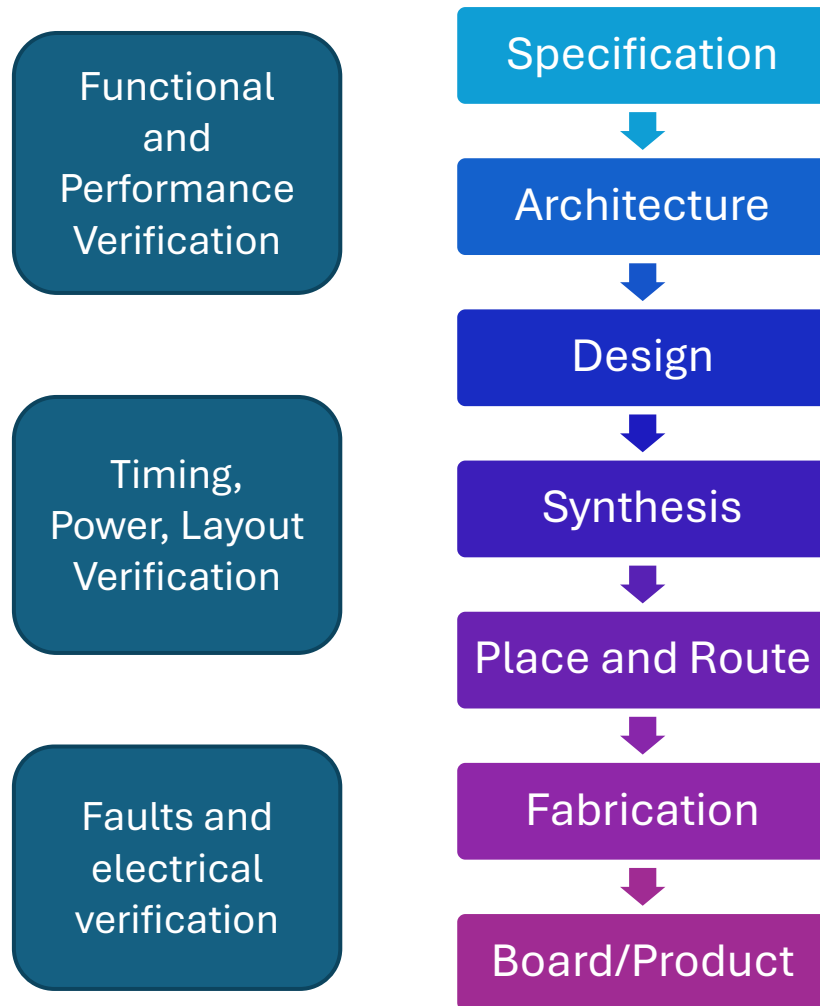
Format	Purpose
.lib	Timing and power (used in synthesis & STA)
.lef	Physical layout info (used in place & route)
.gds	Final layout for fabrication
.db	Binary version of .lib for tools

```
cell(AND2X1) {
  /* gs005 */
  pg_pin(VDD) {
    voltage_name : V;
    pg_type : primary_power;
  }
  pg_pin(VSS) {
    voltage_name : VSS;
    pg_type : primary_ground;
  }
  scaling_factors : AND2X1_factors ;
  area : 7.445 ;
  leakage_power() {
    /* gs006 */
    related_pg_pin : VDD;
    when : "(IN1*IN2)";
    value : 162490.000;
  }
  leakage_power() {
    /* gs006 */
    related_pg_pin : VDD;
    when : "(IN1*IN2)";
    value : 171100.000;
  }
  leakage_power() {
    /* gs006 */
    related_pg_pin : VDD;
    when : "(IN1*IN2)";
```

Technology Library

- Every manufacturing process (40nm, 20nm, 14nm, 7nm, etc) has its own rules for laying out transistors.
- Back-end “library” engineers carefully layout transistors, extract characteristics (like area, delay), corroborate these characteristics with manufactured test chips, and create a library.
- Different variations – SVT (Standard Threshold Voltage), HVT, LVT, HP (High Performance), LP (Low Power), etc.





Design Practices

- Modular design
- Hierarchical design
- Parameterization
- IP based design
- Standardization of interfaces
- Generators

REUSE

Aside: Linting

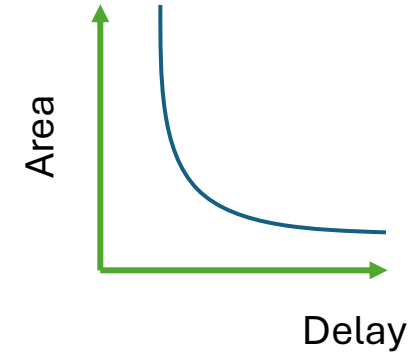
- Static code analysis to identify if the design adheres to pre-defined guidelines
- Examples:
 - Unreachable code
 - Naming conventions
 - Mixing blocking and non-blocking assignments
 - Un-synthesizable constructs
 - Multiple drivers
 - Unintended latches
- Tools: Synopsys Spyglass

Goals of Design

- Area
- Timing
- Performance
- Power
- Security
- Safety/Reliability
- Testability

Tradeoffs

- There is no free lunch!
- Tradeoffs everywhere



- Simplest example:

- Area-Delay Tradeoff

Example: Ripple Carry Adder vs. Carry Lookahead Adder

- Fundamental tradeoff in digital design and computer architecture
 - If you need something to run faster, you have to spend more area
 - If you need something to take less area, it will take more time
 - Area Delay Product is a common metric to compare designs or architectures

Design Optimizations

- **Area**

- **Resource Sharing:** Using the same hardware resources for multiple operations to reduce area.
- **Logic Minimization:** Simplifying logic expressions to reduce the number of gates.

- **Timing**

- **Critical Path Reduction:** Identifying and optimizing the longest path in the circuit to improve overall speed.
- **Retiming:** Adjusting the position of registers to optimize timing and improve performance.

- **Performance**

- **Pipelining:** Breaking down a process into smaller stages with registers between them to increase throughput.
- **Parallelism:** Performing multiple operations simultaneously to speed up processing.

Design Optimizations

- **Power**

- **Clock Gating:** Reducing power consumption by disabling the clock signal to inactive parts of the circuit.
- **Power Gating:** Shutting down entire blocks when not in use to save power.
- **Dynamic Voltage and Frequency Scaling (DVFS):** Adjusting voltage and frequency according to workload requirements.

- **Security**

- **Secure Boot and Encryption:** Ensuring that only authenticated firmware and data are executed or accessed.
- **Hardware Obfuscation:** Making reverse engineering of the design more difficult (e.g., using FPGA fabric).
- **Side-Channel Attack Mitigation:** Designing to resist attacks based on power analysis, timing, or electromagnetic emissions.
- **Access Control Mechanisms:** Restricting access to sensitive parts of the design or memory (e.g., ARM Trustzone).

Design Optimizations

- **Reliability/Safety**

- **Error Detection and Correction:** Implementing mechanisms like parity bits and ECC (Error-Correcting Code) to enhance reliability.
- **Redundancy Techniques:** Using hardware or logic redundancy (e.g., TMR - Triple Modular Redundancy) to tolerate faults.

- **Testability**

- **Design for Testability (DFT):** Incorporating features like scan chains, built-in self-test (BIST), and boundary scan to facilitate easier and more effective testing.

Aside: DFx

- Design For Debuggability (DFD)
- Design For Manufacturability (DFM)
- Design For Testability (DFT)
- Design For Performance Evaluation

Common Digital Systems

- Processors
- Accelerators
- Switches/Routers
- Memories
- I/O Controllers

Common Digital Components

- Datapath Elements: ALUs, multiplexers, registers, buses.
- Control Logic: FSMs, control signals, microcode (if applicable).
- Memory Blocks: Buffers, FIFOs, caches (if present).
- Communication Modules: Interconnects, arbitration logic, protocol handlers.

Design Languages

- Verilog (System Verilog)
 - Most popular
- Several new languages
 - Chisel
 - SpinalHDL
 - PyMTL
 - Amaranth
 - HardCaml
- Typically, companies have their own layers over languages
 - E.g., VIVA at NVIDIA

Let's break the ice!

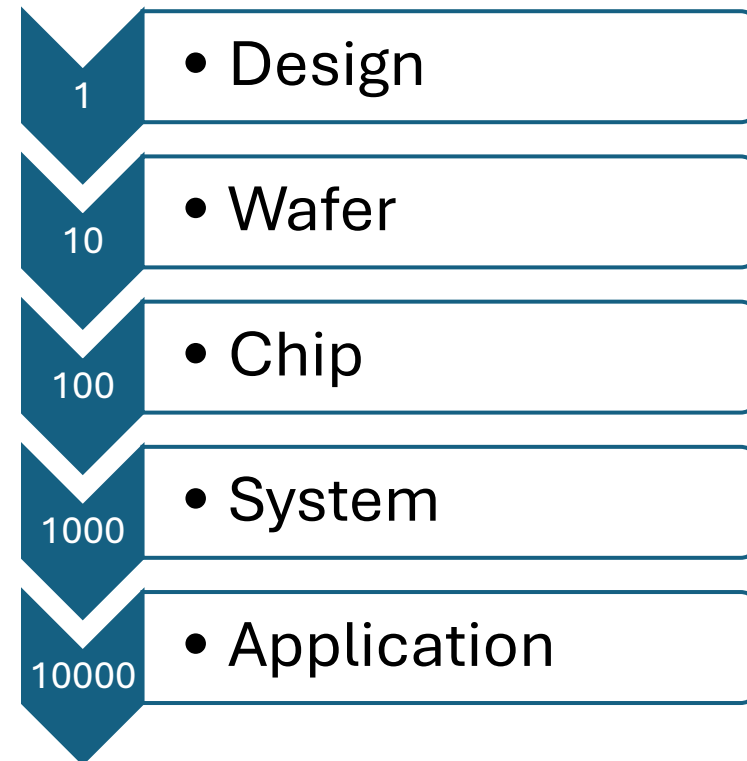
- Pair up with the person
- Introduce yourself
- Say why you're taking this course
- Two Truths and a Lie



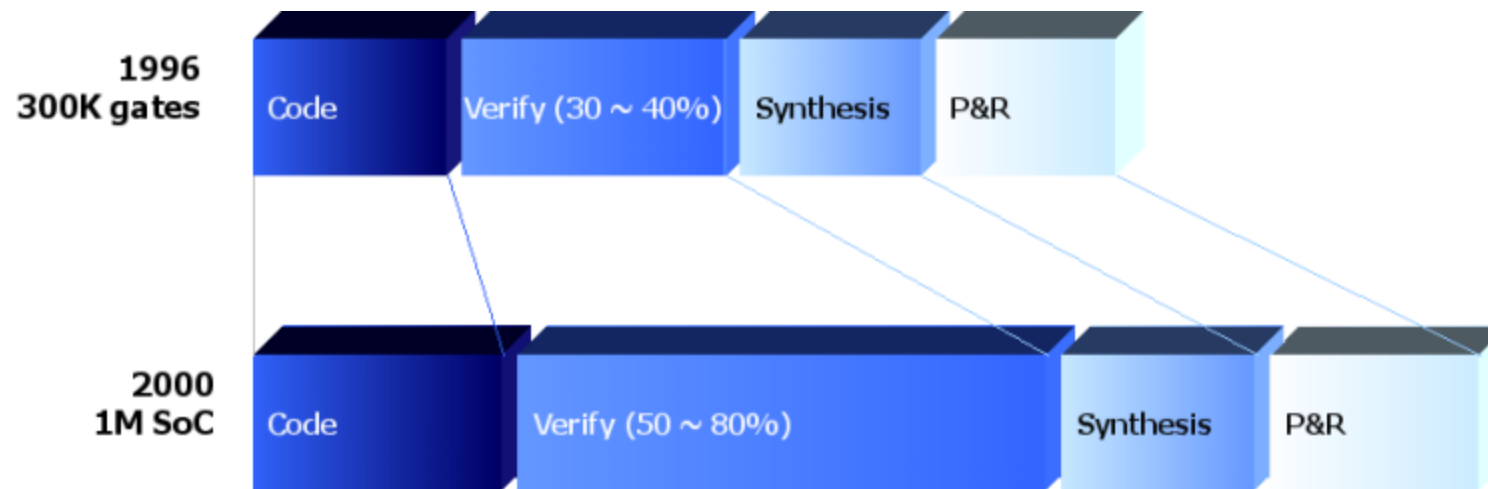
Verification

Why Verification?

- Ensure that the design meets specifications
- Find ‘bugs’ before your customers do!!
- Cost of finding a bug follows the ‘Rule of 10’

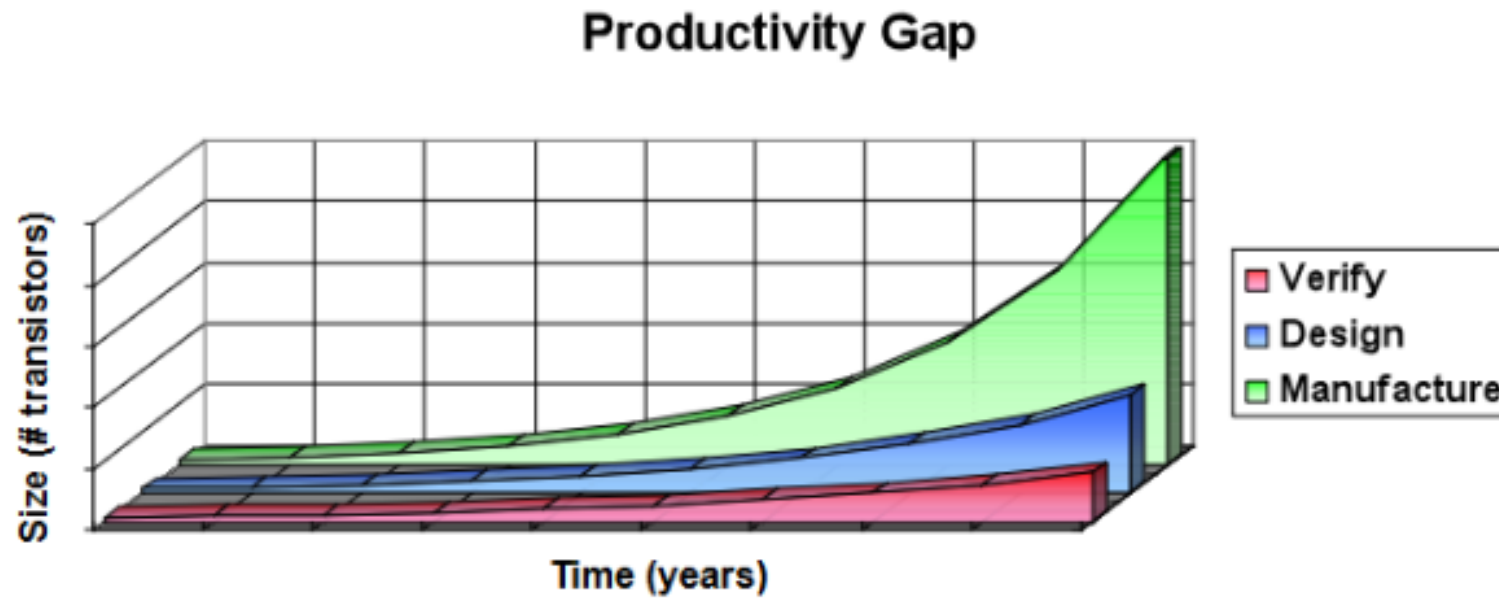


Design Effort



- Over 50% to 80% of design effort is in verification
- About 3-5 verification engineers per design engineer

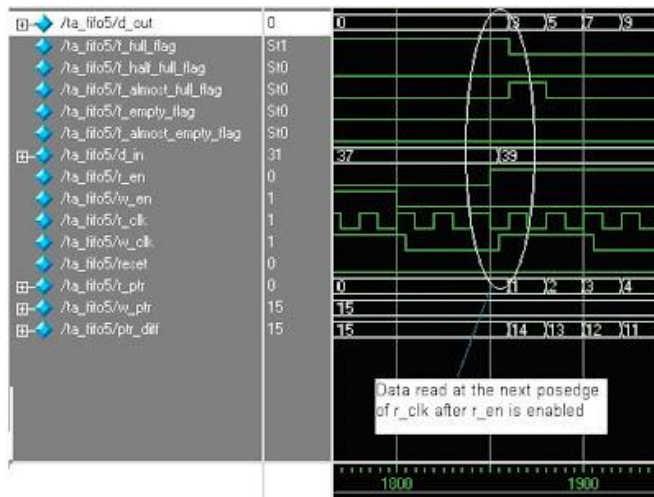
Design and Verification Gap



Types of Verification

- Dynamic verification
 - a.k.a. Simulation-based verification
- Static verification
 - a.k.a. Formal verification

Generally, a combination of both is used

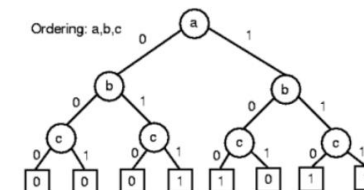


$$S \cap T = \{x \mid x \in S \wedge x \in T\}$$

$$S \cup T = \{x \mid x \in S \vee x \in T\}$$

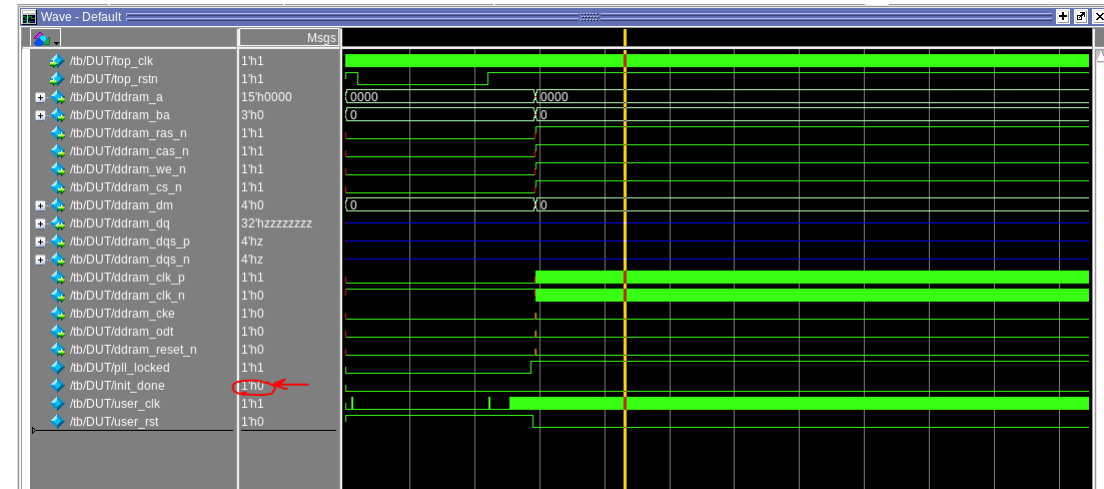
$$S \times T = \{(x, y) \mid x \in S \wedge y \in T\}$$

$$(a + b + c)(a + \bar{e} + f)(\bar{c} + \bar{d} + g) \dots$$

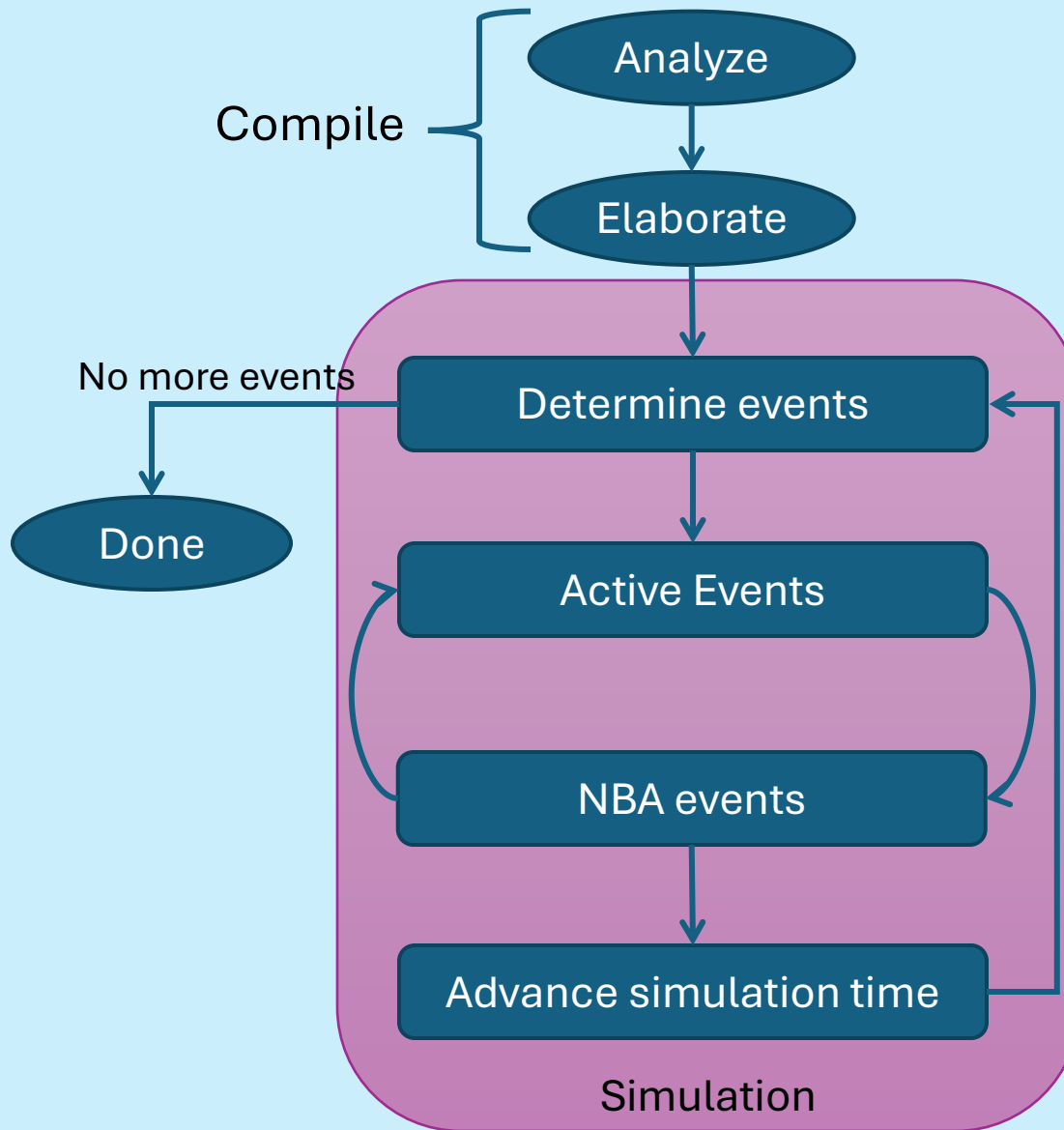


Simulation

- Applying inputs and observing outputs
 - Dumps waves and load them in a viewer
- Examples from the top-3 EDA vendors:
 - Siemens ModelSim/QuestaSim
 - Synopsys VCS
 - Cadence Xcelium
- Simulation semantics described in the language's LRM (Language Reference Manual)
- A vendor can implement a simulator in any way, but must adhere to the LRM



Aside: How does a simulator work?



Event handling:

- Evaluate
- Update
- Schedule

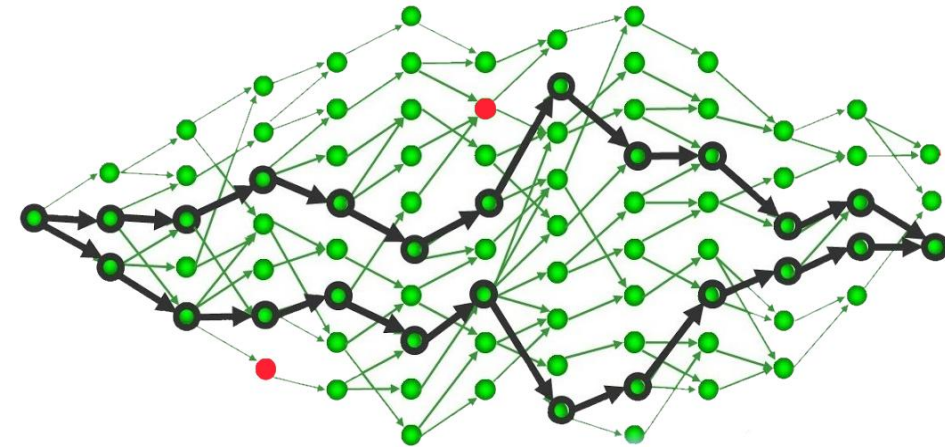
Limitations of Simulation

- Simulation is very slow
 - Simulation speed scales with the complexity of the design
 - Size (gate count)
 - Abstraction (gate-level vs. RTL)
 - Note: Simulation time vs. Wall clock time ('5 ns' in 5 minutes)
- The effectiveness of simulation lies in the capability of the verification engineer to think of testcases/scenarios
 - It is almost impossible to exercise the entire state space
 - Random vectors can help!

Formal Verification

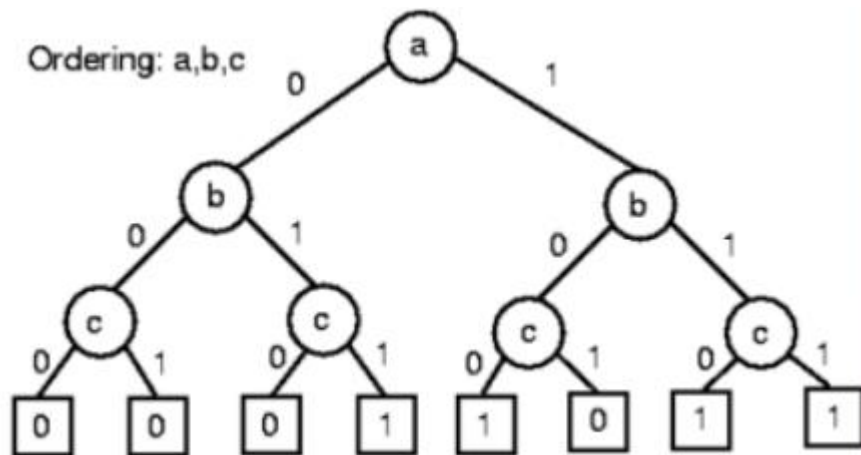
Not a focus of this course

- Formal verification applies mathematics to solve the verification problem
- Represent your design in the form of a mathematical formula/equation and establish that it is correct (meaning that it holds under specified constraints) by using proofs
- Exhaustive; subject to the state explosion problem
- Does not scale well. It becomes impossible to do for even fairly complicated designs

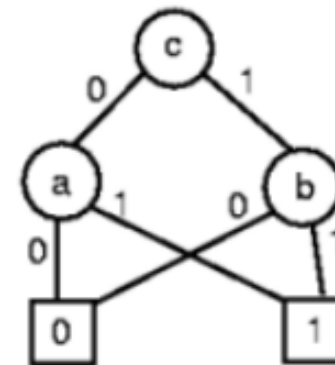


Binary Decision Diagrams

- Data structure used to represent and manipulate boolean functions
- Reduced, Ordered BDDs (ROBDDs) are canonical
- Example: $F = a'bc + ab'c' + abc' + abc$



Reduced, Ordering: c,a,b



Satisfiability

- Check if a boolean function can be true (satisfied)
- Steps:
 - Represent the function in POS (Product-Of-Sums) form or CNF (Conjunctive Normal Form)
 - Try to find a satisfying assignment – an assignment which makes the expression evaluate to 1
 - A boolean formula is **satisfiable** if atleast one assignment is satisfying
 - A boolean formula is **unsatisfiable** if no assignment is satisfying

- Example:

$$(v+w+x')(x+y'+z)(v'+w'+y')(u+v+y)(u+v'+x)$$

One satisfying assignment is $u=1, v=1, w=0, x=1, y=0, z=1$

Temporal logic

- BDDs and SAT are applicable to combinatorial logic
- We need something to model sequential behavior
- Temporal logic can help
- CTL (Computation Tree Logic), CTL*, LTL (Linear Temporal Logic)
- Temporal logic let us express the behaviors/properties of designs in an analyzable form
- Example:
When a request **req** occurs, it will eventually be acknowledged by **ack**
 $AG(\mathbf{req} \rightarrow F \mathbf{ack})$

Formal Verification Techniques

Equivalence Checking

- Compare two different representations/models
- Typically, the two different levels of abstraction like RTL and gates, or RTL and C++
- Very popular in the industry
- Eg. RTL vs. synthesized netlist (not ‘typical’ design verification)

Model Checking (or Property Checking)

- Check whether a given model (design) satisfies a given property under given constraints
- Various efficient algorithms for checking temporal logic formulas on finite state models
- Usually used to aid simulation-based design verification

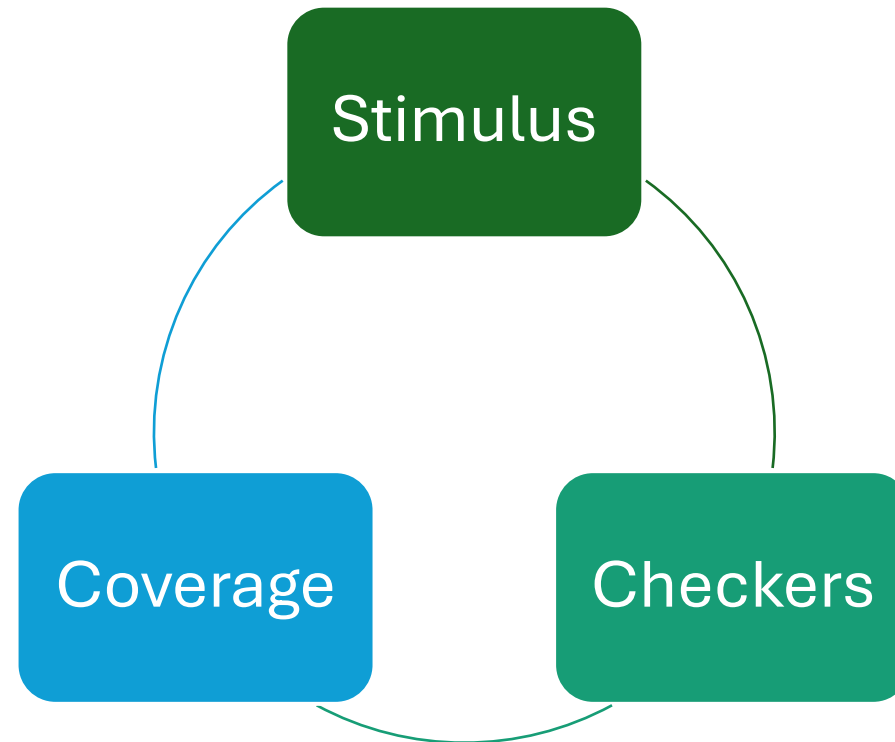
Theorem Proving

- Relationship between a specification and an implementation is regarded as a theorem and is proved using symbolic representations, mathematical logic, and inductive proofs
- Mostly academic tools available
- Can handle infinite state systems

Simulation



The trifecta of verification



Stimulus



- Applying inputs to stimulate the DUT
- Types:
 - Active: Drive values into the DUT
 - Reactive: Respond to something that happened in the DUT
- Directed tests: **Good and bad**
- **Randomization** is great for finding unexpected bugs
 - Use **constraints** to prevent applying illegal input values
 - What can you randomize?
 - Values (eg: Different values of Data, Addr, etc)
 - Commonly, distribution of values is more important than just the values themselves
 - Values you randomize can be inputs to the DUT or the state of the DUT
 - Timings and order (eg: Pkt1 -> delay -> Pkt2 -> etc)
 - How many interfaces to wiggle?
 - Etc...

Checkers

- Ensuring that the DUT is behaving correctly
- Flag an error when something illegal happens
 - **“Always On” checkers**
- Signal level checking
 - Assertions
 - Specify behavior that the design must or must not exhibit
 - Examples
 - A FIFO should not overflow or underflow
 - **assert property** (@(posedge clk) start ##2 transfer);
 - **assert property** (@(posedge clk) !(grant0 & grant1));
- Behavioral checking
 - Scoreboards
 - Example: A txn was sent into the DUT with data X, so the response txn should have data Y
 - Register checkers, State machine checkers,...

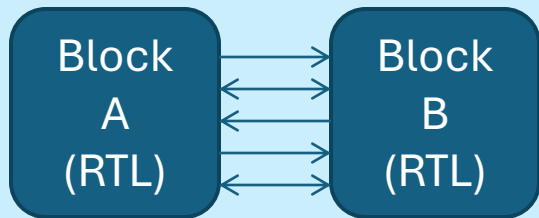
Coverage

- Quantitative measure of how well did we exercise the DUT
 - Run all test, merge coverage results and generate a score
 - Higher the coverage score, higher the quality of the stimulus
 - Remember to ignore failing tests when collecting coverage
- Ensure that the DUT was exposed to all possible (read: *interesting*) scenarios
- Coverage metrics
 - **Code** Coverage
 - Line Coverage
 - State Machine Coverage
 - Toggle Coverage
 - Branch Coverage
 - **Limitations?**
 - **Functional** Coverage
 - Express scenarios you want to see in the form of code

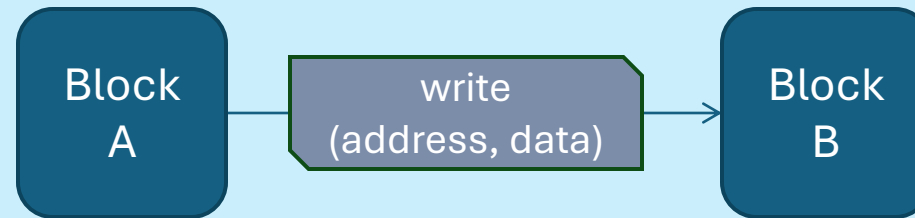
Aside: Transaction Level Modeling

- Model digital systems at higher levels of abstraction
 - More abstract than RTL
- Transaction Level Modeling (TLM)
 - Less details
 - Easier to code
 - Less time to simulate

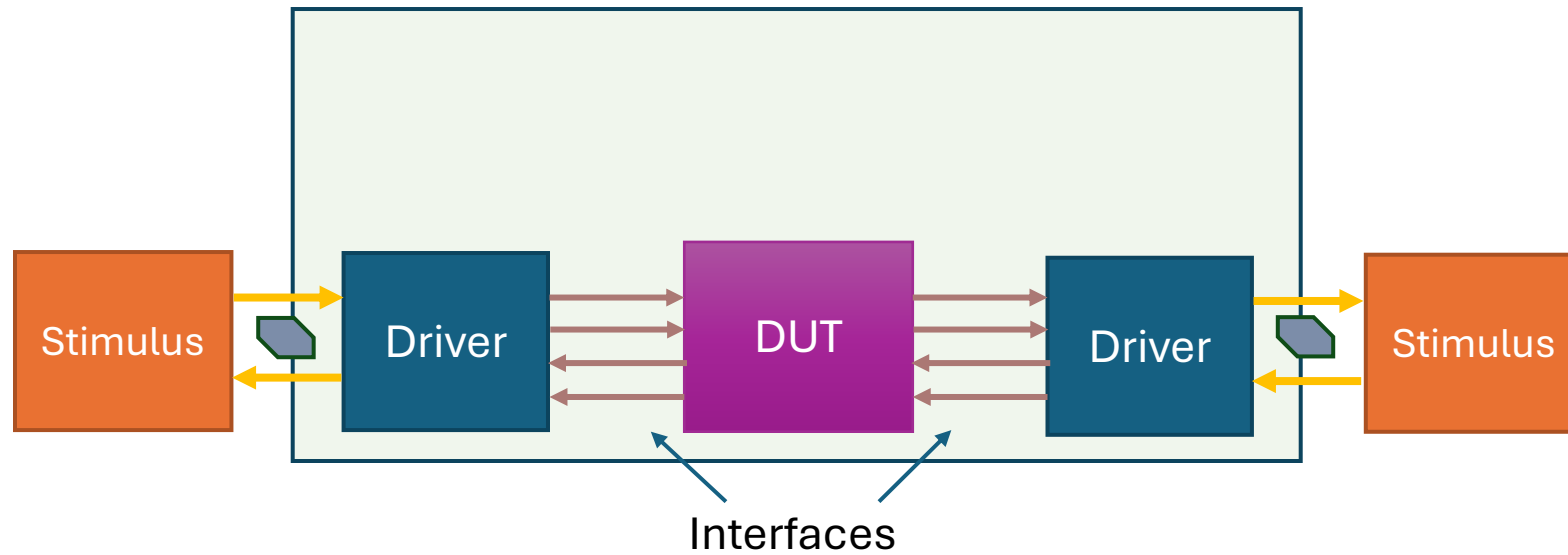
Traditional RTL based approach



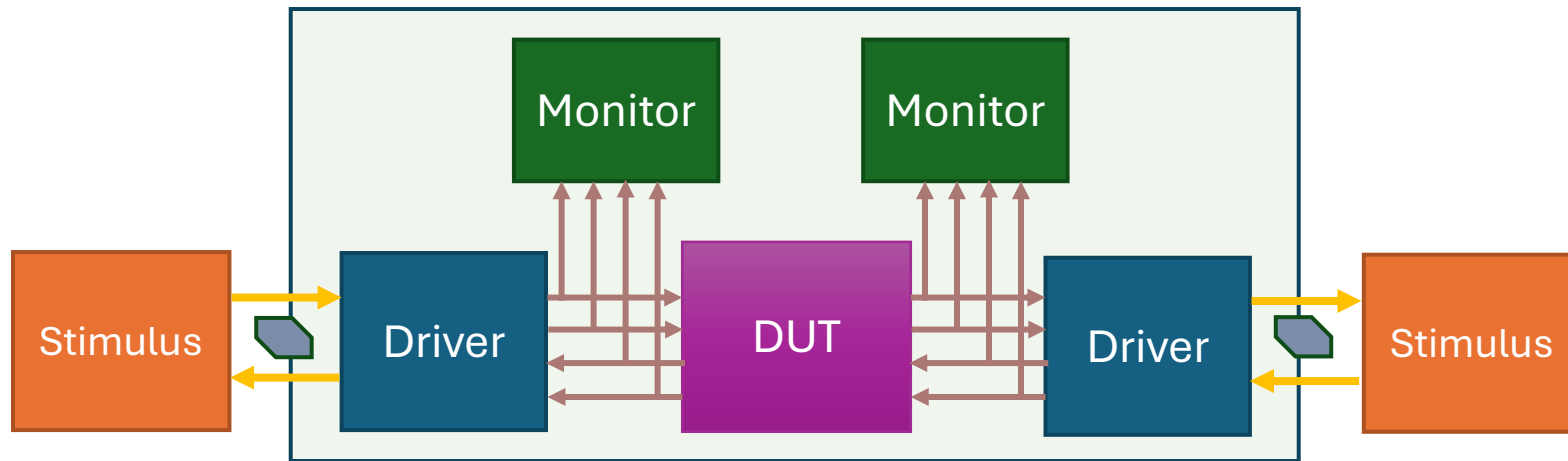
New TLM based approach



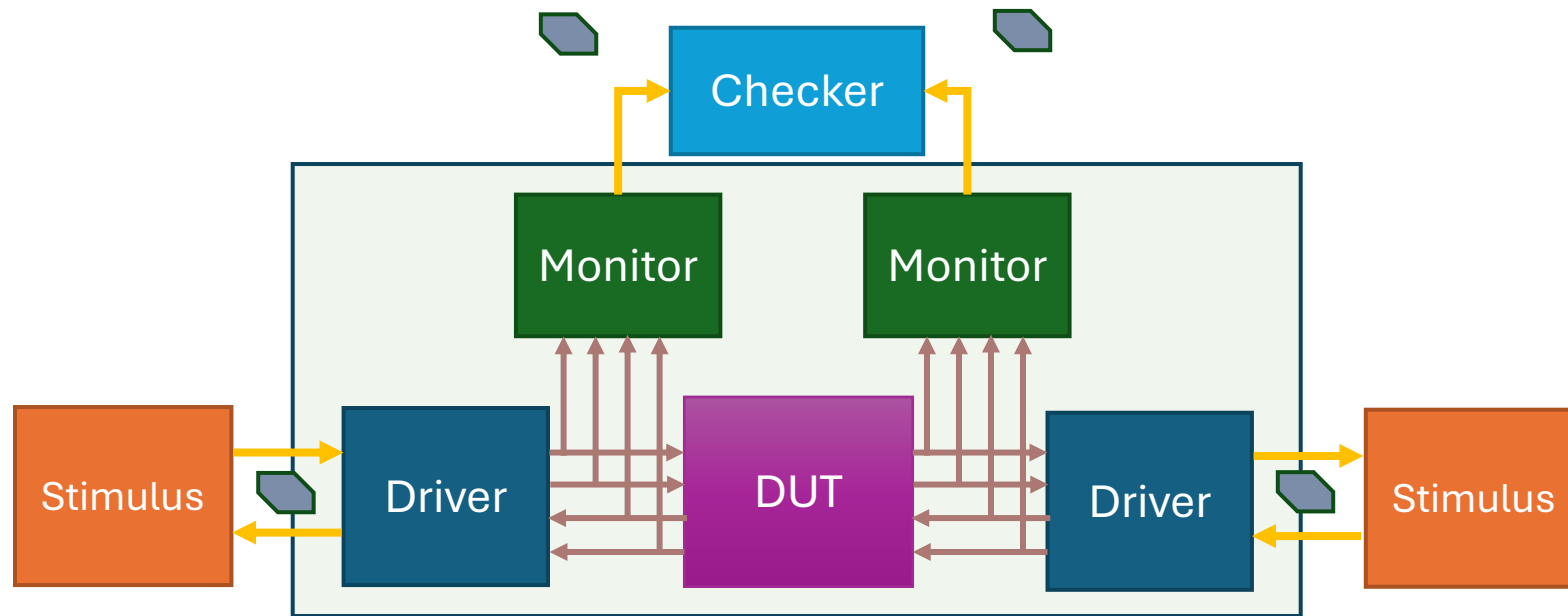
Testbench



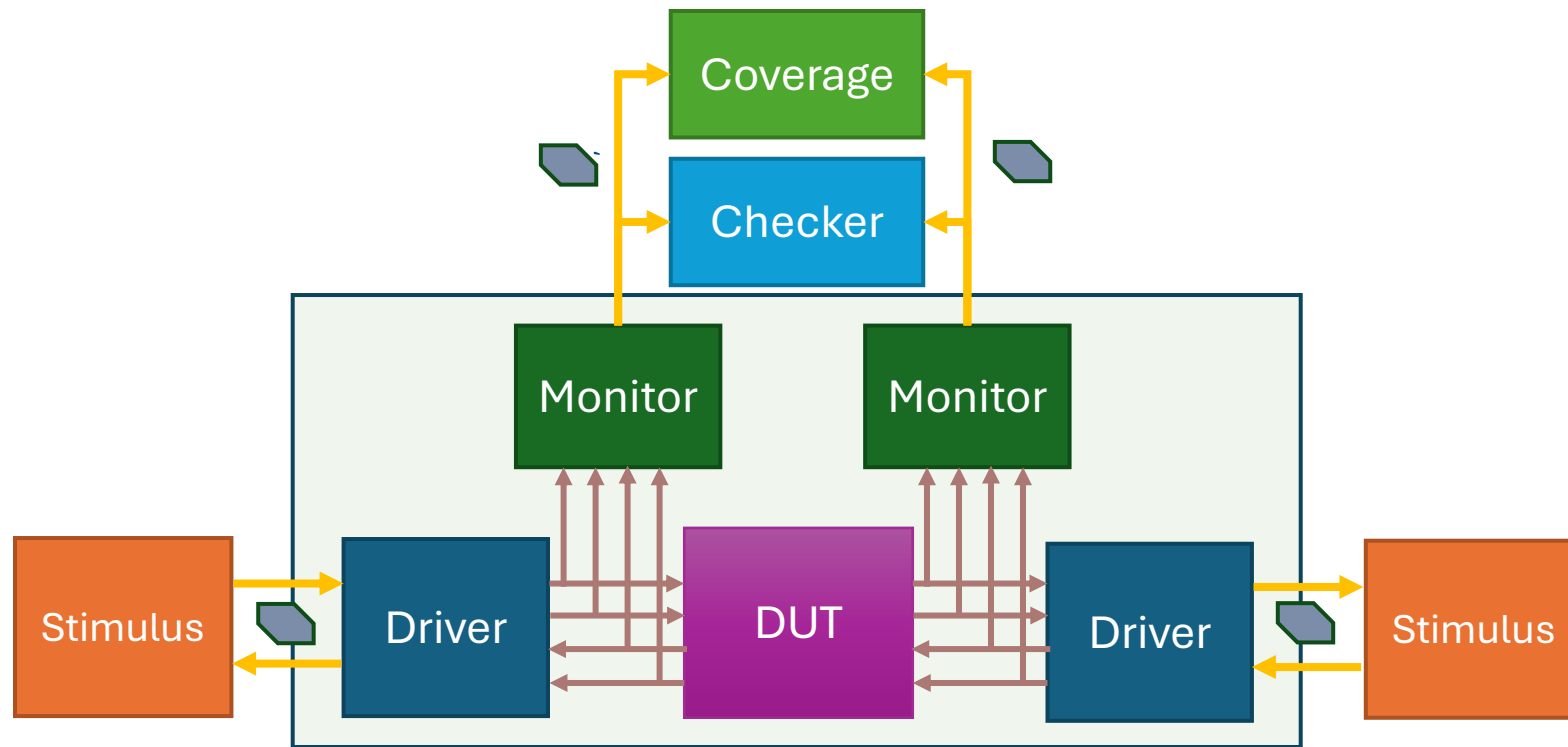
Testbench



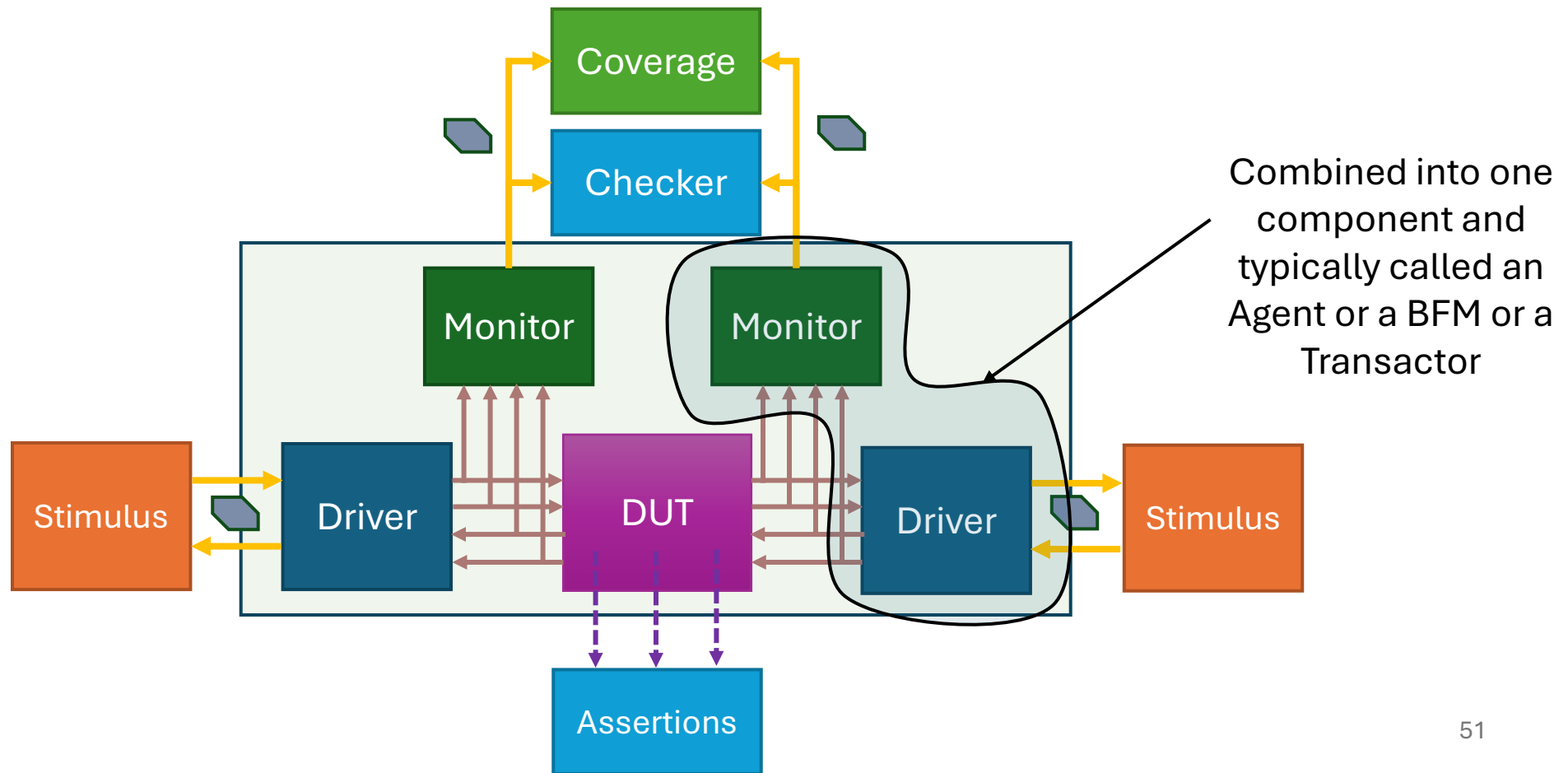
Testbench



Testbench



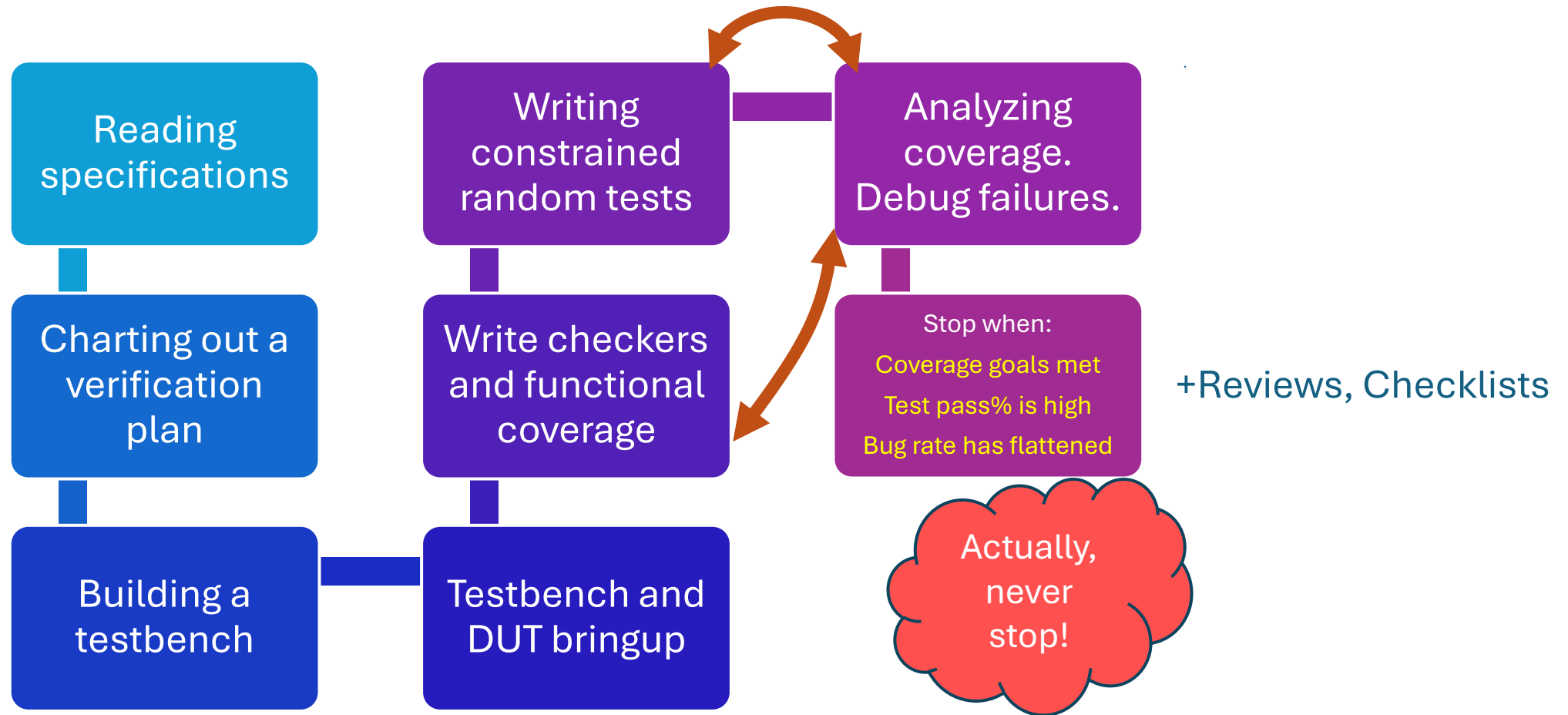
Testbench



Verification Plan

- A document that describes how will you verify a DUT
- Strategy
 - Methodology
 - Simulation, Formal
 - Testbench components
 - Sign-off metrics
- Features
 - For every feature:
 - Stimulus (“Apply inputs $A \rightarrow B \rightarrow C$, keeping $D=0$ ”)
 - Checking (“Check that E becomes 1 every time we enter state X”)
 - Coverage (“State X was reached”)
- Multiple formats – Word, Excel, Synopsys HVP, Cadence VPlan

Life of a Verification Engineer



Aside: Separation of concerns

- Process of separating a system into distinct parts that adhere to a single and unique purpose
- Stimulus from checking
 - Self-checking stimulus is not “always-on”
- Coverage from stimulus
 - Common pitfall: “I wrote the stimulus. I know it is covering these scenarios”
 - Same stimulus may achieve different coverage if design or testbench changes; e.g. a FIFO depth or a clock frequency change
- Testbench from stimulus
 - Common pitfall: Make testbench component specific to a DUT
- Separate engineers working on different aspects
 - Stimulus, Checkers, Coverage
 - **Architecture, Design, Verification**

Verification Languages

- Why are HDLs not sufficient?
 - Programming-world features like classes, enums, etc
 - Native support for assertions, randomization, coverage
- Popular verification languages
 - SpecMan e
 - OpenVera
 - **SystemVerilog**
 - Think of it as Verilog++
 - IEEE 1800

Verification Methodologies

- Standardization is the key
- Reuse is a very important aspect
 - You will hear the term “VIP” (Verification IP)
 - E.g. Buy a checker from an EDA vendor and plug-in into our testbench
- Some standards
 - VMM (Verification Methodology Manual)
 - OVM (Open Verification Methodology)
 - **UVM (Universal Verification Methodology)**



Challenges in verification

- Designs are getting larger
 - Cycle times are getting smaller
 - Tools are not scaling accordingly
- Specification capture and documentation
- Use case identification
- Security verification
 - Verifying that a chip does nothing it is NOT supposed to do
- Safety verification
 - Is your system safe in case of a failure?

<https://ieeexplore.ieee.org/document/8000621>

Closing the verification gap...



Reuse



Abstraction



Acceleration



Methodology

Review Questions

- What is the role of a technology library in the synthesis process?
- What are some common goals of digital design, and how do they conflict with each other?
- What is static timing analysis, and why is it “static”?
- How does simulation differ from formal verification?
- What are the three pillars of verification?
- What are the limitations of code coverage, and how does functional coverage address them?